Introducción al Software Testing.

Asesoría: Planeación y diseño de pruebas.

Prof. Ulises Vélez Saldaña

12 de octubre de 2026





AyD



2022 Año de Ricardo Flores Magón
PRECURSOR DE LA REVOLUCIÓN MEXICANA

Desarrollo de un Sistema De Información para La Toma de Decisiones

Introducción al Software Testing.

Asesoría: Planeación y diseño de pruebas.

Prof. Ulises Vélez Saldaña

12 de octubre de 2026





AyD



2022 Año de Ricardo Flores Magón
PRECURSOR DE LA REVOLUCIÓN MEXICANA

1. Pruebas y Calidad de software
   - ¿Que es la Calidad?
   - Calidad de software
   - Modelos de calidad de software
2. Fundamentos de las pruebas
3. Más fundamentos de las pruebas
4. ¿Que son las pruebas?
5. Principios de las pruebas



Calidad

- La calidad es una propiedad inherente de cualquier cosa que permite que la misma sea valorada con respecto a cualquier otra de su misma especie.



Calidad

- La calidad es una propiedad inherente de cualquier cosa que permite que la misma sea valorada con respecto a cualquier otra de su misma especie.
- **Producción:** La calidad es el grado en que un producto cumple con las especificaciones del diseño.



Calidad

- La calidad es una propiedad inherente de cualquier cosa que permite que la misma sea valorada con respecto a cualquier otra de su misma especie.
- **Producción:** La calidad es el grado en que un producto cumple con las especificaciones del diseño.
- **Usuario:** La capacidad de satisfacer los deseos de los consumidores.



Calidad

- La calidad es una propiedad inherente de cualquier cosa que permite que la misma sea valorada con respecto a cualquier otra de su misma especie.
- **Producción:** La calidad es el grado en que un producto cumple con las especificaciones del diseño.
- **Usuario:** La capacidad de satisfacer los deseos de los consumidores.

Definición Formal (ISO 9000)

la calidad es el grado en el que un conjunto de características inherentes a un objeto (producto, servicio, proceso, persona, organización, sistema o recurso) cumple con los requisitos.



Factores de la calidad

- Dimensión técnica: engloba los aspectos científicos y tecnológicos que afectan al producto o servicio.
- Dimensión humana: son las relaciones entre clientes y empresas.
- Dimensión económica: intenta minimizar costos tanto para el cliente como para la empresa.

Para ello se debe establecer la cantidad justa y deseada de producto que hay que fabricar y que se ofrece.



- ISO 12207 – Modelos de Ciclos de Vida del Software,
- Norma ISO/IEC 9126, para evaluar productos de software.
- Estándar ISO/IEC 14598, marco de trabajo para evaluar la calidad de los productos de software.
- ISO 25000:2005 (SQuaRE -Software Quality Requirements and Evaluation)
- SPICE Norma Internacional para la Evaluación de Procesos de Software.
- CMMI Modelo de mejora continua
- IEEE (Institute of Electrical and Electronics Engineers)
- PSP y TSP Mejora de trabajo individual y en equipo.
- Moprosoft NyCE-NOM, alineada con IEEE, ISO y CMMI
- iSTQB International Software Testing Qualification Board.



1 Pruebas y Calidad de software

2 Fundamentos de las pruebas
- ¿Que son las pruebas?
- Ejemplos de importancia de las pruebas
- Error, Defecto y Fallo
- ¿Para que probar?
- Costo de los defectos

3 Más fundamentos de las pruebas

4 ¿Que son las pruebas?

5 Principios de las pruebas



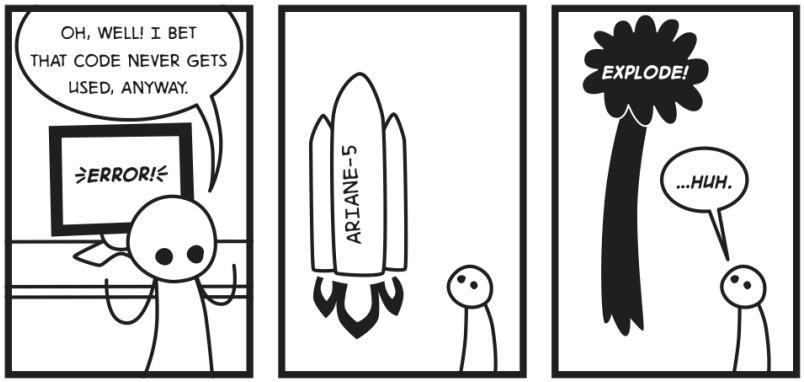
Pruebas y Calidad

- Las pruebas y revisiones aseguran la mejora de la calidad de productos de software, así como de los procesos de desarrollo.
- Las pruebas son solamente una parte del proceso de calidad.
- Probar software no es fácil debido a su naturaleza abstracta y falta de materia prima.
- El software no pude ser desarrollado en una línea de producción.



Fallos provocados por defectos en software

- Vuelo 501, 4 de junio de 1996, prueba de vuelo del sistema desechable del airline 5. El cohete se destruyó después de 37 segundos después del lanzamiento, uno de los defectos más caros de software de la historia.



Fallos provocados por defectos en software

- Vuelo 501, 4 de junio de 1996, prueba de vuelo del sistema desechable del airline 5. El cohete se destruyó después de 37 segundos después del lanzamiento, uno de los defectos más caros de software de la historia.
- Pacientes reciben una dosis letal de radiación por la Therac 25 1985 a 1987.



Palabras clave: error, defecto y fallo

Un error humano puede introducir un defecto en el código y provocar que este falle.

- Error: Acción humana que produce un resultado incorrecto.
- Defecto: Desperfecto en un componente o sistema que puede causar que el componente o sistema falle en desempeñar las funciones requeridas.
- Fallo: Manifestación física o funcional de un defecto. Si un defecto es encontrado durante la ejecución de una aplicación esta puede producir un fallo.

Causas del error

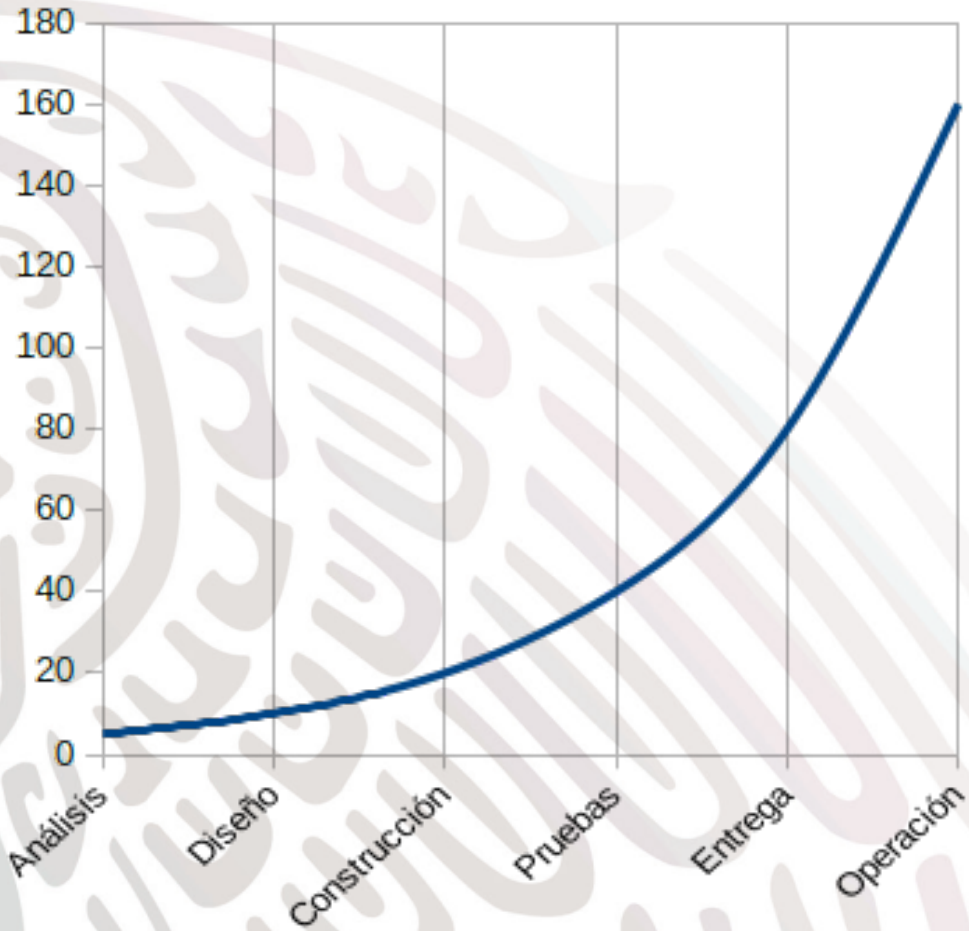
Factores humanos (plazos, complejidad, distracciones, agotamiento) o condiciones ambientales (magnetismo, contaminación, fallos de discos duros, fallos energéticos, etc.)

¿Por que es importante probar?

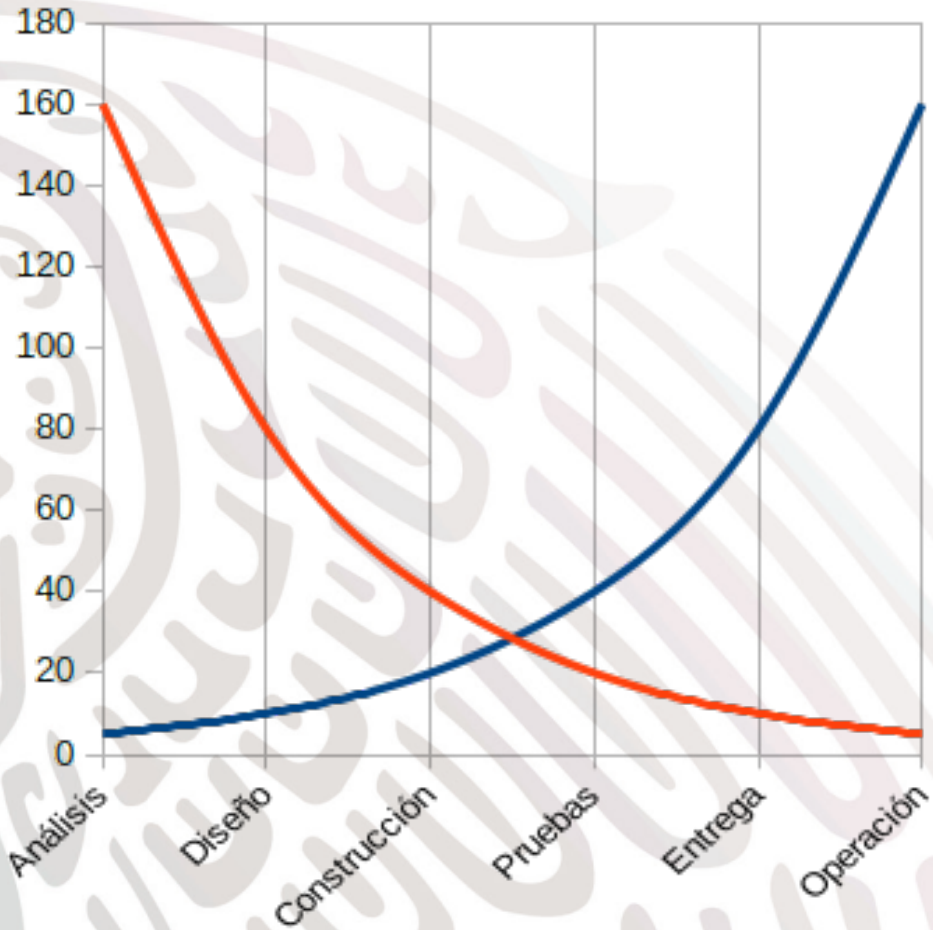
- Mejora de la calidad de un producto de software. Que el producto adquiera los atributos deseados.
- Reducción del riesgo de detectar errores. Reducir el riesgo en la operación del sistema.
- Satisfacer compromisos. Requisito obligatorio por parte del cliente sobre el cumplimiento de normas legales.



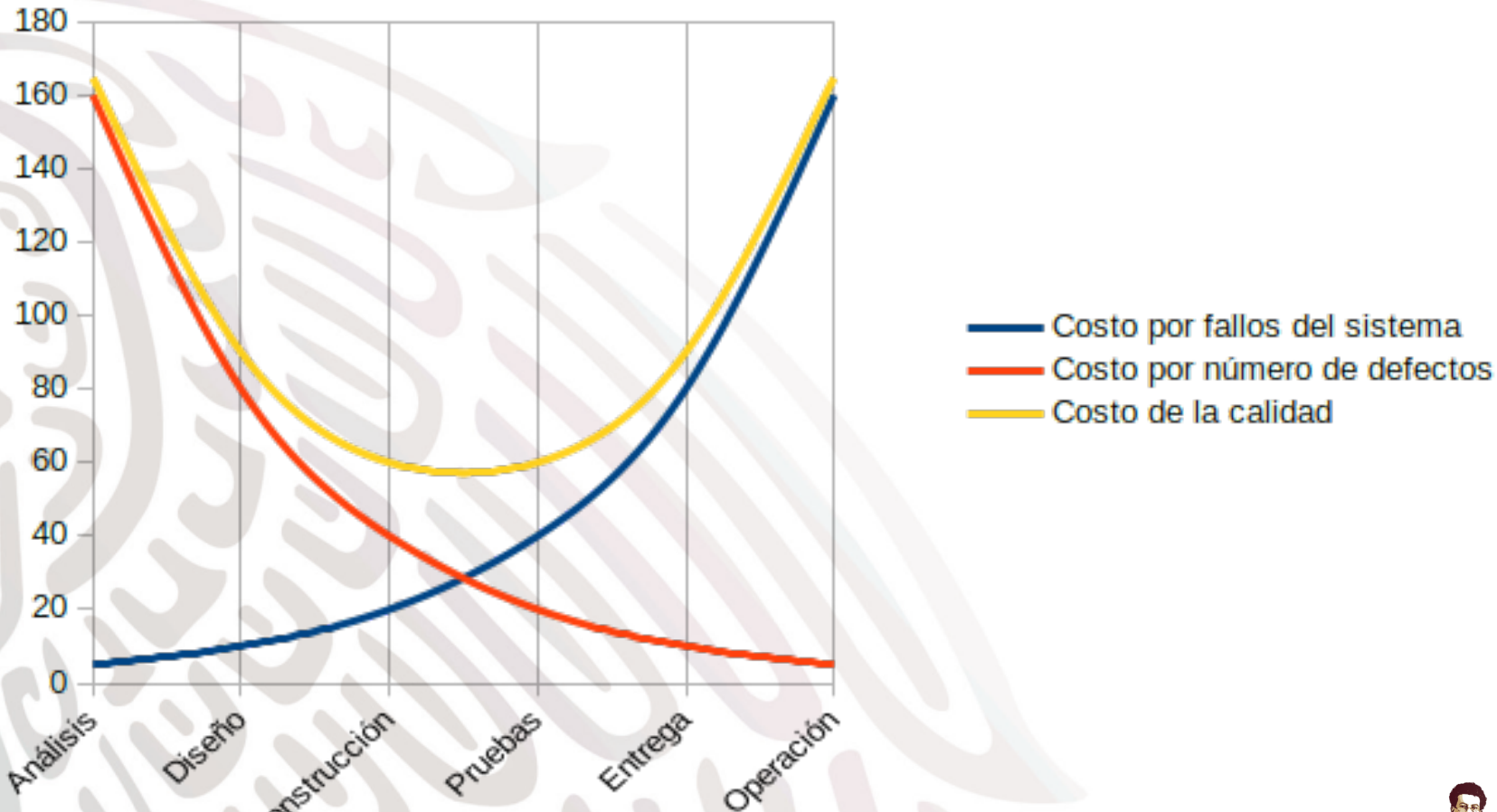
Costos de la calidad



Costos de la calidad



Costos de la calidad



1 Pruebas y Calidad de software

2 Fundamentos de las pruebas

3 Más fundamentos de las pruebas
- Definiciones
- Que abarca la calidad para un software
- Tipos de Aseguramiento de la calidad
- Atributos funcionales de la calidad
- Atributos no funcionales de la calidad
- Objetivos de las pruebas

4 ¿Que son las pruebas?

5 Principios de las pruebas



Definiciones

- **Software:** Programas, procedimientos y posiblemente documentación y datos pertenecientes a la operación de un sistema basado en una computadora.



Definiciones

- **Software:** Programas, procedimientos y posiblemente documentación y datos pertenecientes a la operación de un sistema basado en una computadora.
- **Calidad de Software:** La totalidad de la funcionalidad y prestaciones de un producto de software que están relacionados con su capacidad de satisfacer las necesidades explícitas o implícitas.



Definiciones

- **Software:** Programas, procedimientos y posiblemente documentación y datos pertenecientes a la operación de un sistema basado en una computadora.
- **Calidad de Software:** La totalidad de la funcionalidad y prestaciones de un producto de software que están relacionados con su capacidad de satisfacer las necesidades explícitas o implícitas.
- **Calidad:** Grado en el cual un componente, sistema o proceso satisface requisitos especificados y/o necesidades y expectativas del usuario o cliente.



¿Que se debe probar de un software?

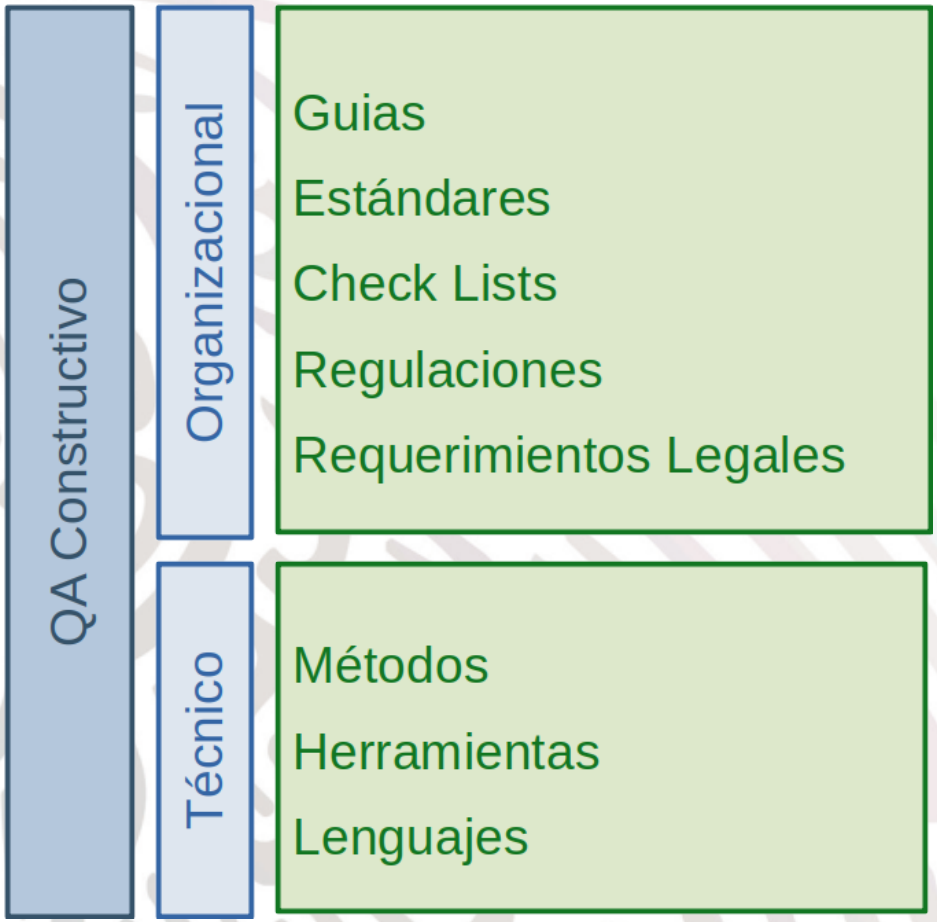
De acuerdo a la ISO/IEC 9126 la calidad comprende:

- Funcionalidad
- Fiabilidad
- Usabilidad
- Eficiencia
- Mantenibilidad
- Portabilidad

Tipos de aseguramiento de la calidad

Actividades Constructivas: que tienen como finalidad prevenir defectos por ejemplo aplicando métodos, técnicas o usando herramientas adecuadas.
Actividades Analíticas: Con el objeto de detectar defectos mediante la ejecución de pruebas para corregir los defectos.

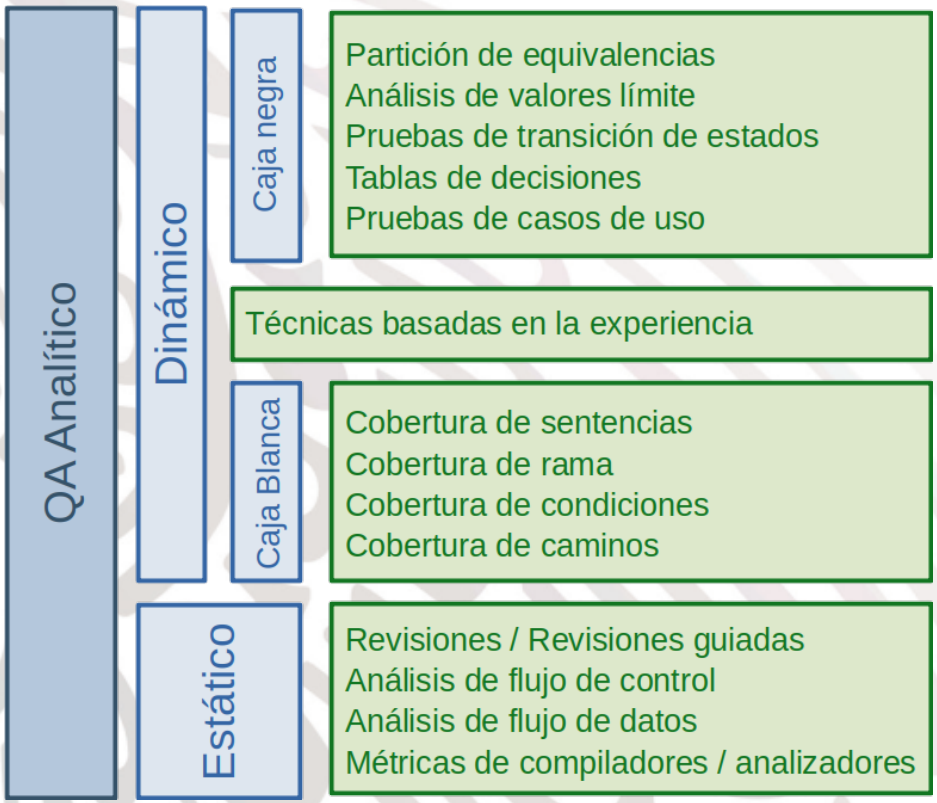
Aseguramiento de la calidad constructiva



Consigna:

- Los defectos evitados no requieren ser reparados.
- Los defectos introducidos en el pasado no deben ser repetidos.
- Los defectos deben prevenirse.

Aseguramiento de la calidad analítica



Consigna:

- Los defectos deben ser detectados tan pronto como sea posible respecto del proceso.
- **Pruebas estáticas:** Son pruebas sin la ejecución del programa.
- **Pruebas dinámicas:** Incluye la ejecución del programa.



Atributos funcionales de la calidad

- En ingeniería de software, Funcionalidad se refiere a:
  - **Corrección**: La funcionalidad satisface los atributos / capacidades
  - **Completitud**: La funcionalidad satisface todos los requisitos (funcionales).
- Según la ISO/IEC 9126 debe incluir:
  - **Adecuación** (Suitability): Adecuado para ser usado en el contexto.
  - **Exactitud** (Accuracy).: mapeo de las necesidades del negocio hacia los modelos de programación
  - **Interoperabilidad** (interoperability): capacidad de interactuar con otros procesos y sistemas.
  - **Seguridad** (Security): El sistema está protegido.
  - **Cumplimiento de funcionalidad*** (Functional compilance): Capacidad de adherirse a estándares, normas y regulaciones.



Atributos no funcionales

- **Fiabilidad** (Reliability):
  - Madurez, Tolerancia a fallos, Recuperabilidad, Cumplimiento de fiabilidad.
  - En determinadas condiciones, el software mantendrá su capacidad a lo largo de un periodo de tiempo.
- **Usabilidad** (Usability):
  - Comprensibilidad, Aprendabilidad, Operabilidad, Atractivo, Cumplimiento de usabilidad.
  - Fácil de usar, fácil de aprender, conforme a normas, uso intuitivo.
- **Eficiencia** (Efficiency):
  - Comportamiento del sistema: funcionalidad y respuesta temporal.
  - El sistema requiere la utilización de un mínimo de recursos para ejecutar una tarea determinada.



Atributos no funcionales

- **Mantenibilidad** (Maintainability):
  - Analizabilidad, Modificabilidad, Estabilidad, Testabilidad, Cumplimiento de mantenibilidad.
  - Medida del esfuerzo requerido para realizar cambios en los componentes de un sistema.
- **Portabilidad** (Portability):
  - Adaptabilidad, Instalabilidad, Co-existencia, Reemplazabilidad, Cumplimiento de portabilidad.
  - Capacidad del software de ser transferido a un nuevo entorno.
  - Fácil de instalar y desinstalar.



Aspectos de los atributos de la calidad

- Algunos Atributos se influyen mutuamente.
- Esta dependencia determina el objetivo de las pruebas.
- Por ejemplo Eficiencia vs Portabilidad.
- Se deben realizar distintos tipos de prueba con el objeto de medir distintos tipos de atributos.



Objetivos para realizar pruebas

- **Adquirir conocimiento sobre los defectos en un objeto de prueba** (test object). Los defectos contenidos en un objeto de prueba deben ser detectados y descritos de tal forma que se facilite su corrección.
- **Conformidad de la funcionalidad**. La funcionalidad del sistema debe ser implementada tal y como ha sido especificada.
- **Generar información**. Un sistema que ha sido probado de forma adecuada se considera que cumple con la funcionalidad esperada y cuenta con un alto nivel de calidad.
- Ganar confianza.



¿Que tanto debemos probar?

- **¿Cuantas pruebas son suficientes?**
  - Criterios de salida (exit criterial)
  - No encontrar más defectos es un criterio apropiado para finalizar las actividades de pruebas. Sin embargo es necesario utilizar otras métricas para reflejar de forma adecuada el nivel de calidad alcanzado.
- **¿Que son las pruebas basadas en riesgo?** (risk based testing). El nivel de riesgo determina el grado en el cual se ha probado, es decir: responsabilidad en caso de fallos, probabilidad de la ocurrencia de fallos, aspectos relativos a factores económicos y propios del proyecto.
- **¿Que son las pruebas basadas en plazos y presupuestos?** (time and budget testing). La disponibilidad de recursos: el personal, tiempo o presupuesto pueden determinar la medida del esfuerzo del proceso de pruebas.



Algunos conceptos finales

- **Caso de prueba** (test case): La definición de un caso de prueba debe incluir:
  - Precondiciones.
  - Conjunto de valores de entrada.
  - Postcondiciones esperadas.
  - Identificador único.
  - Dependencia de otros casos de prueba.
  - Referencia al requisito que se atiende.
  - Forma en la cual debe ejecutarse el caso de prueba y certificar los resultados.
  - Prioridad.
- **Base de pruebas** (test base): Conjunto de documentos que definen los requisitos de un componente o sistema. Utilizado como fundamento para el desarrollo de casos de prueba.



Algunos conceptos finales

- **Código** o **código fuente**: Programa de ordenador escrito en un lenguaje de programación que puede ser leído por las personas.
- **Depuración** (debugging): Localización y corrección de defectos en el código fuente.
- **Desarrollo de software** (software development): Secuencia de actividaes cuyo objeto es desarrollar un sistema basado en una computadora.
- **Requisito** (requirement): Describe un atributo deseado o considerado obligatorio.
- **Revisión** (Review): Evaluación de un producto o del estado de un proyecto para detectar discrepancias respecto de los resultados planificados y para recomendar mejores.



1 Pruebas y Calidad de software

2 Fundamentos de las pruebas

3 Más fundamentos de las pruebas

4 ¿Que son las pruebas?
- ¿Que significa probar?
- Objetivos de las pruebas
- Pruebas y depuración

5 Principios de las pruebas



Probar significa más que solo ejecutar pruebas

El proceso de pruebas incluye:

- Planificación y control.
- Selección de condiciones de pruebas.
- Diseño y ejecución de casos de prueba.
- Comprobación de resultados.
- Generación de informes respecto del proceso de pruebas y el sistema sujeto a pruebas.
- Finalización y completar el cierre.

La revisión de documentos, código fuente y la realización de análisis estático también ayudan a prevenir la aparición de defectos en el código.

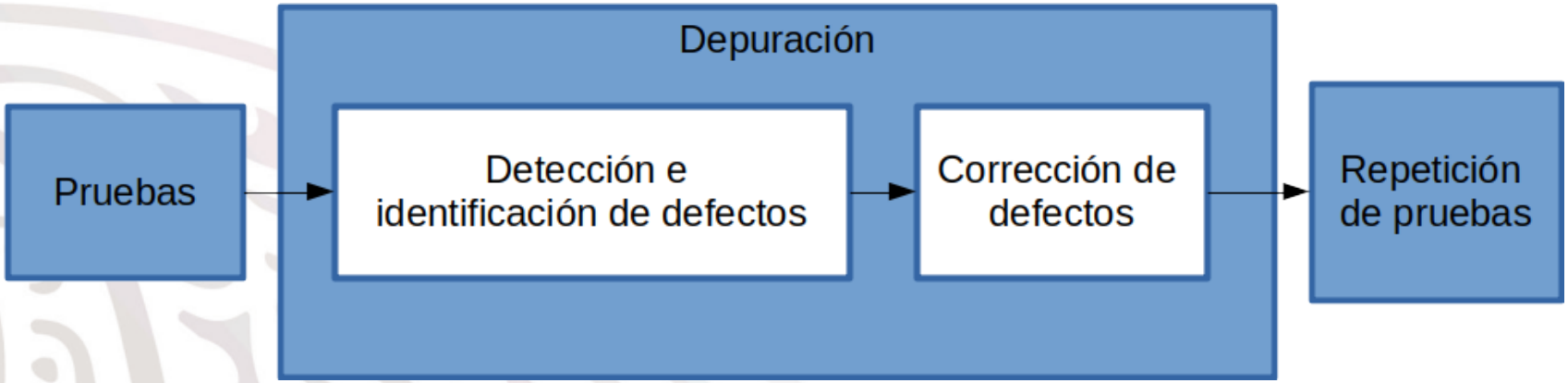


Objetivos de las pruebas

- Detección de defectos.
  - Pruebas de desarrollo: Causar tantos fallos como sea posible.
  - Pruebas de aceptación: Para confirmar que el sistema funciona como se esperaba.
- Generación de confianza respecto al nivel de calidad.
- Aportación de información para la toma de decisiones.
- Prevención de defectos.



Pruebas y Depuración



- Probar y Repetir las pruebas (Re-testing) son actividades propias del proceso de pruebas. la repetición de pruebas verifican que los defectos han sido corregidos.
- la depuración y corrección de defctos son actividades del proceso de desarrollo. Los desarrolladores pueden reproducir los fallos, analizar el estado del programa y detectar el defecto correspondiente con el objeto de corregir.



1 Pruebas y Calidad de software

2 Fundamentos de las pruebas

3 Más fundamentos de las pruebas

4 ¿Que son las pruebas?

5 Principios de las pruebas
- Principio 1 y 2
- Principio 3 y 4
- Principio 5 y 6
- Principio 7



Los 7 principios de las pruebas



Principio 1: El proceso de pruebas demuestra la presencia de errores

- El proceso de pruebas puede probar la presencia de defectos.
  - Las desviaciones identificadas a lo largo del proceso de pruebas demuestran la presencia de un fallo.
  - La causa de un fallo puede no ser obvia.
- El proceso de pruebas no puede demostrar la ausencia de defectos.
  - Las pruebas reducen la probabilidad de la presencia de defectos que permanezcan sin ser detectados. La ausencia de fallos no demuestra que el producto de software esté libre de errores.
  - El mismo proceso de pruebas puede contener errores.
  - Las condiciones de pruebas pueden ser inapropiadas para detectar errores.



Principio 1

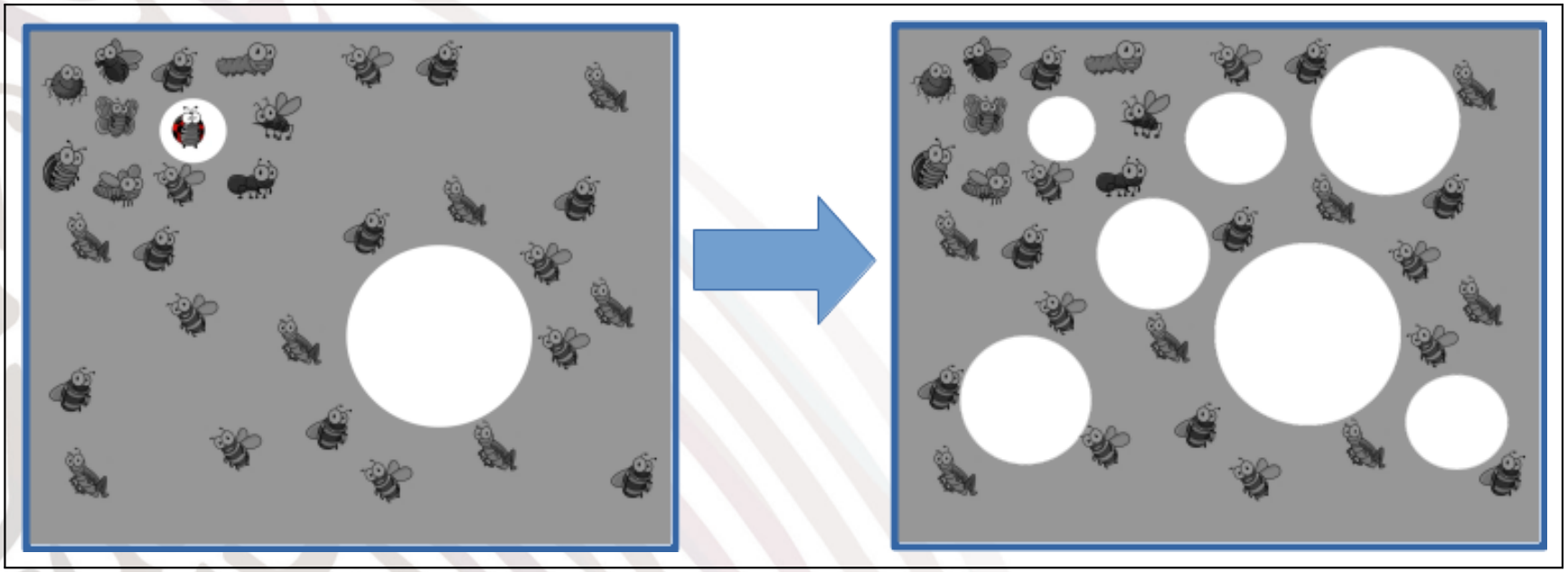


Figura: Principio 1: El proceso de pruebas demuestra la presencia de errores

Principio 2: No es posible realizar pruebas exhaustivas

- Pruebas exhaustivas (Exhaustive testing)
  - Enfoque de pruebas donde el conjunto de pruebas abarca todas las combinaciones de valores de entrada y precondiciones.
- Explosión de casos de prueba (Test case explosion)
  - Define el incremento exponencial de esfuerzo y costo en el caso de pruebas exhaustivas.
- Prueba de muestra (Sample test)
  - La prueba incluye solamente a un subconjunto (generado de manera sistemática o aleatoria) de todos los posibles valores de entrada.
  - En condiciones reales, se utilizan generalmente pruebas de muestra. Probar todas las combinaciones posibles de entradas y precondiciones solo es económicamente viable en casos triviales.



Principio 3: Pruebas tempranas

- Cuanto más temprana es la detección de un defecto, menos costosa es su corrección
  - Se obtiene una máxima rentabilidad cuando los errores son corregidos antes de la implementación.
  - Los conceptos y especificaciones también pueden ser probados.
  - Los defectos detectados en la fase de concepción son corregidos con los menores esfuerzos y costos.
- La preparación de una prueba también consume tiempo
  - El proceso de pruebas implica más que solo la ejecución de pruebas
  - Las actividades de pruebas pueden ser preparadas antes de que el desarrollo se haya completado-
  - las actividades de pruebas (incluidas las revisiones) deben ser ejecutadas en paralelo a la especificación y diseño de software.



Principio 3

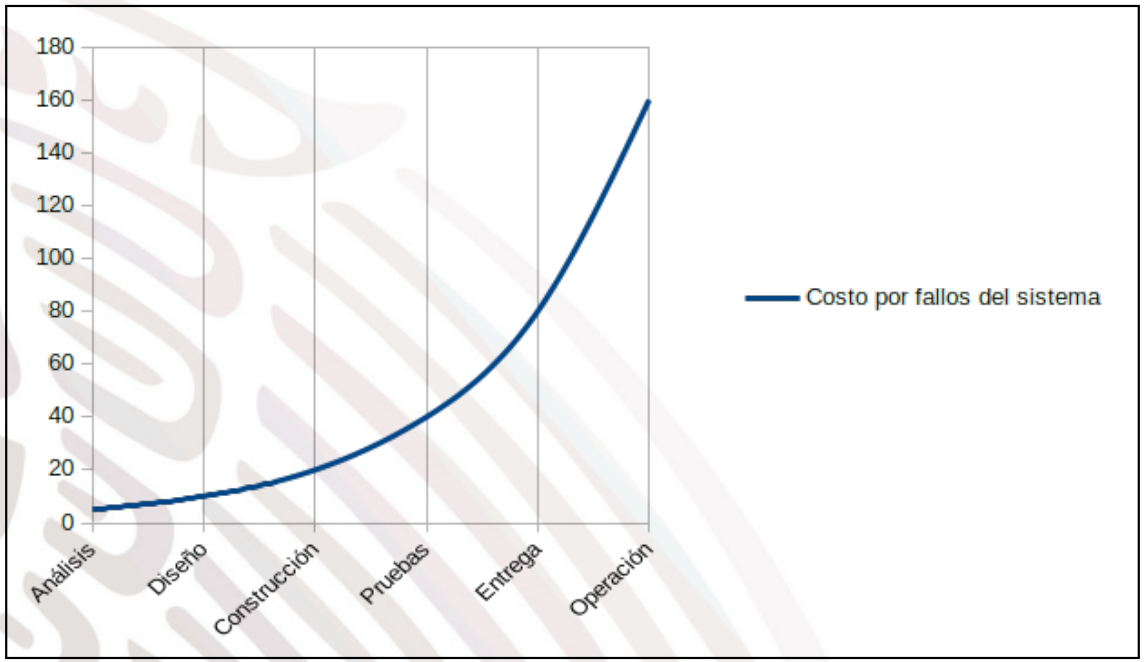


Figura: Principio 3: Pruebas tempranas

Principio 4: Agrupamiento de defectos

- Encuentre un defecto ¡Y encontrará más defectos "cerca"!
  - Los defectos aparecen agrupados como hongos o cucarachas.
  - Vale la pena investigar un mismo módulo donde se ha detectado un defecto.
- Los probadores deben ser flexibles
  - Habiendo sido detectado un defecto, es conveniente volver a considerar el rumbo de las pruebas posteriores.
  - La identificación/localización de un defecto puede ser investigada con un mayor grado de detalle, por ejemplo, realizando pruebas adicionales o modificando pruebas existentes.



Principio 4

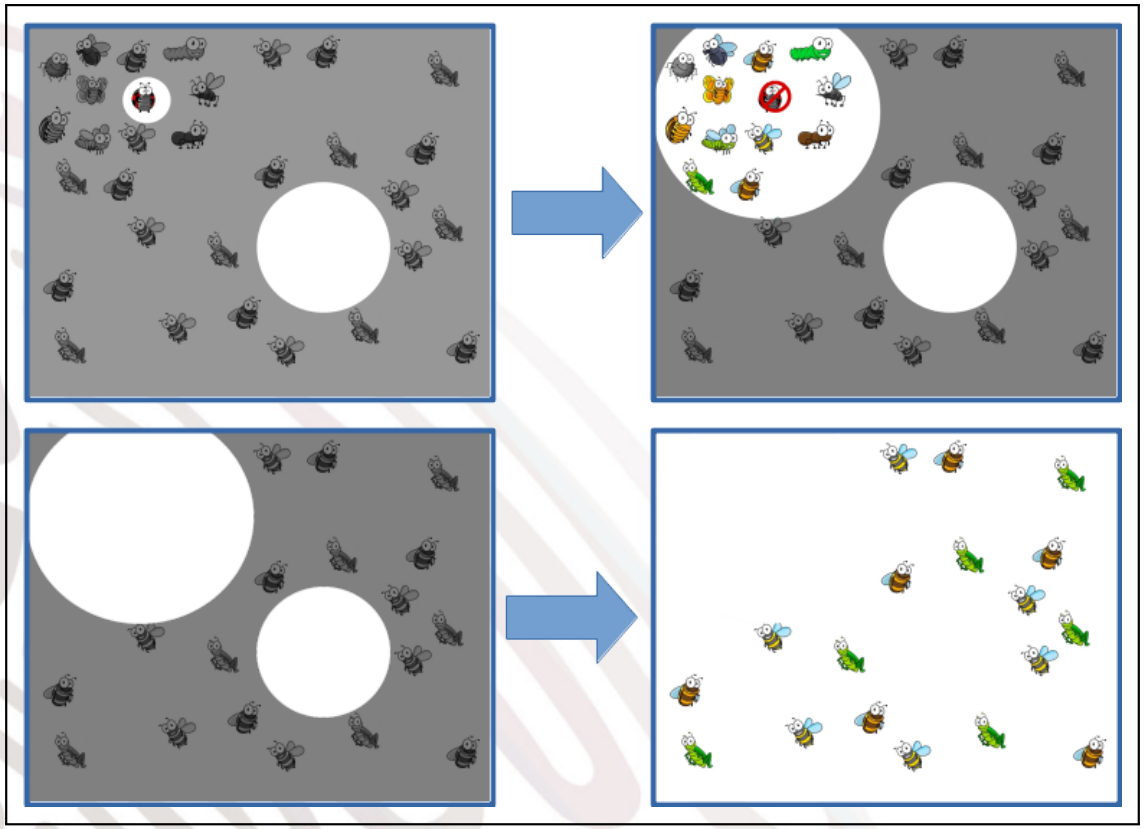


Figura: Principio 4: Agrupamiento de defectos

Principio 5: Paradoja del pesticida

- Repetir pruebas en las mismas condiciones no es efectivo.
  - Cada caso de pruebas debe contar con una combinación única de parámetros de entrada para un objeto de prueba particular, de lo contrario no se podrá obtener información adicional.
  - Si se ejecutan las mismas pruebas de forma reiterada no se encontrarán nuevos defectos.
- Las pruebas deben ser revisaras/modificadas regularmente para los distintos módulos de código.
  - Es necesario repetir una prueba tras una modificación del código: corrección de defectos, nueva funcionalidad.
  - La automatización de pruebas puede resultar conveniente si un conjunto de casos de prueba se debe ejecutar frecuentemente.



Principio 5

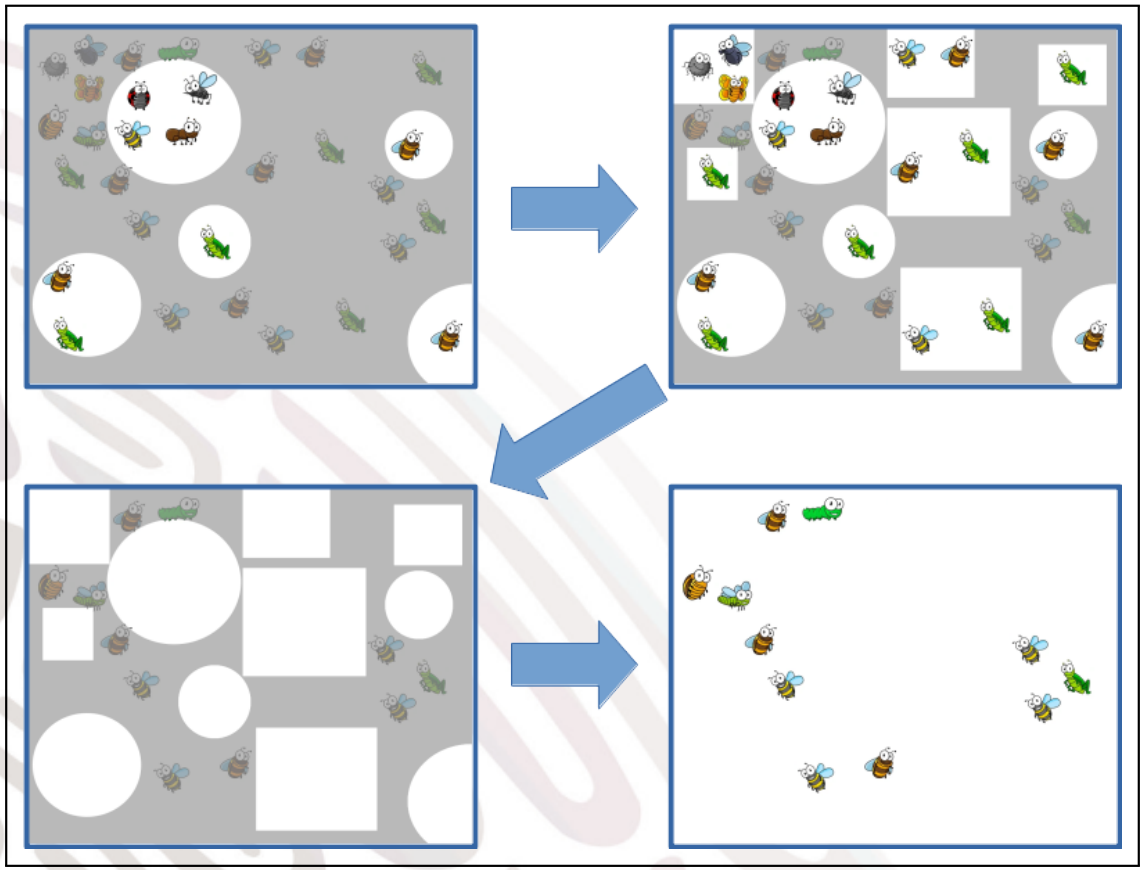


Figura: Principio 5: Paradoja del pesticida

Principio 6: Las pruebas dependen del contexto

- Las pruebas se realizan de forma diferente en diferentes contextos
  - Los objetos de prueba pueden ser ejecutados de forma diferente.
  - El controlador del motor de un coche requiere pruebas diferentes respecto de aquellas para una aplicación de e-commerce.
- Entorno de pruebas vs entorno de producción (test environment, test bed vs prduction environment)
  - Las pruebas tienen lugar en un entorno distinto del entorno de producción. El entorno de pruebas debe ser muy similar al entorno de producción.
  - Siempre habrá desviaciones entre el entorno de pruebas y el entorno de producción. Estas desviaciones ponen en tela de juicio las conclusiones que se obtuvieran tras las pruebas.



Principio 7: La falacia de la ausencia de errores

- Un proceso de pruebas adecuado detectará los fallos más importantes.
- En la mayoría de los casos el proceso de pruebas no detectará todos los defectos del sistema, pero los defectos más importantes deberían ser detectados.
- Esto por sí solo no prueba la calidad del software.
- La funcionalidad del software puede no satisfacer las necesidades y expectativas de los usuarios.
- No se puede introducir la calidad a través de las pruebas, ella tiene que construirse desde el principio.



Principio 7

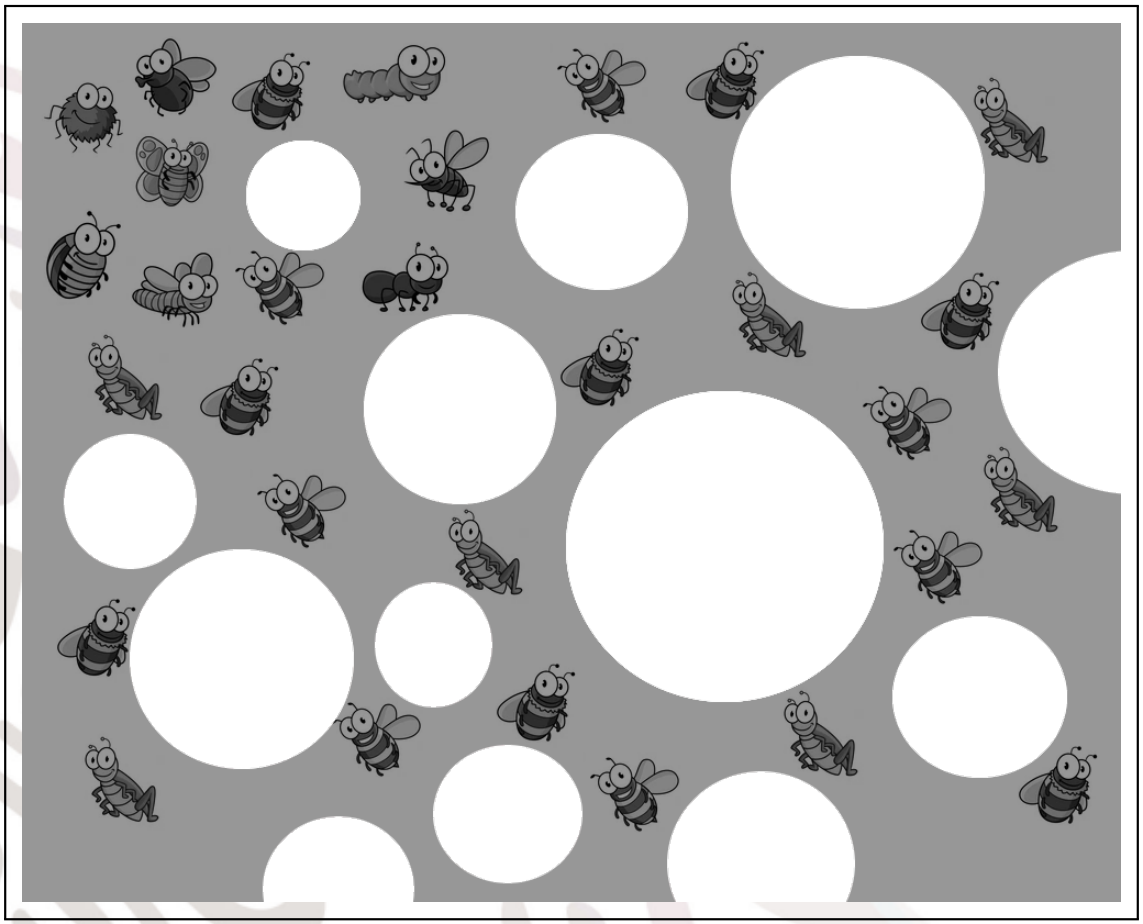


Figura: Principio 7: La falacia de la ausencia de errores